DEVOPS
BUILDING & BUILD TOOLS

Detailed Notes • Special Focus on Apache Maven

Core idea: DevOps aims to make software delivery faster, repeatable and reliable. The build stage converts source code into a usable artifact and can automate compilation, testing, packaging and other tasks.

Group Name	Khushi Kumari
Group No.	7
Members	Khushi Kumari Mahak Kataria Harshita Sahu Hrishita Patel Kratika Jain Ishika Gupta Heena Choudhary
Branch	CSE
Subject	DevOps Practices and Principles
Topic	Building and Its Tools — Focus on Maven

Build Better • Test Earlier • Automate More • Deliver Reliably

Contents

How to use these notes: Read the concept first, then learn the Maven lifecycle, POM, dependencies and commands. The final pages are designed for viva and exam revision.

1. Building in DevOps

In software development, building means processing source code and project resources to produce software that can be tested, packaged or delivered. In DevOps, the build process is usually automated so that the same steps can be repeated consistently.

A simple build pipeline

SOURCE	COMPILE	TEST	PACKAGE	ARTIFACT	DEPLOY
code	bytecode	test results	JAR/WAR	versioned file	environment

What happens during a build?

- Source files are read from the project.
- Required dependencies are resolved.
- Source code is compiled when compilation is required.
- Automated tests can be executed.
- Files are packaged into a distributable artifact.
- Reports or additional checks can be generated.
- The resulting artifact can be passed to later DevOps stages.

Build vs. Deploy

Build	Deploy
Creates or prepares the software artifact.	Moves/releases the artifact to a target environment.
May compile, test and package code.	May configure servers, containers or cloud environments.
Example: mvn package	Example: a deployment pipeline publishing a JAR/container.

Exam line: A DevOps build is an automated and repeatable process that converts source code into a testable or distributable software artifact.

2. Why Build Tools Are Needed

Without a build tool, developers may have to manually compile source code, download libraries, run tests, copy files and create packages. Build tools provide automation and a standard way to perform these activities.

Need	How a build tool helps
Automation	Runs repetitive build steps using commands/configuration.
Dependency management	Downloads and tracks required libraries and versions.
Consistency	The same build process can be used by multiple developers.
Testing	Automates execution of configured tests.
Packaging	Creates standard artifacts such as JAR/WAR.
CI/CD integration	Build commands can run on automated servers.
Reproducibility	A defined build configuration reduces differences between machines.

Manual approach vs automated approach

Manual: Download library → configure classpath → compile → test → package → repeat.

Build tool: Define project configuration → run a command → tool performs the configured lifecycle steps.

Benefits in a team

- New team members can understand the standard project structure faster.
- CI servers can execute the same commands used by developers.
- Dependency versions are recorded with the project.
- Failures become easier to reproduce because build steps are explicit.

Important: Build automation is one part of DevOps. A complete DevOps pipeline also includes source control, CI, testing, security, deployment, monitoring and feedback.

3. Types of Build Tools

Different ecosystems use different build and package-management tools. The correct tool depends on the programming language, framework and project requirements.

Tool	Common use	Key idea
Maven	Java / JVM	Convention-based lifecycle, POM and dependency management.
Gradle	Java / JVM / Android	Flexible, programmable build system.
Ant	Java	Task-based build automation using XML configuration.
npm	JavaScript / Node.js	Package management and project scripts.
MSBuild	.NET	Build engine used widely in Microsoft/.NET projects.
CMake	C/C++	Build-system generation and project configuration.

Maven vs Gradle — basic comparison

Maven	Gradle
Uses XML-based POM configuration.	Uses a programmable build configuration.
Strong convention and standard lifecycle.	Highly flexible and customizable.
Common in enterprise Java projects.	Common in modern JVM and Android ecosystems.
Easy to understand for standardized builds.	Useful when custom build logic is important.

Focus of this assignment: Maven is especially important because it combines project management, dependency management and build automation around a standard lifecycle.

4. Introduction to Apache Maven

Apache Maven is a build automation and project management tool primarily used for Java projects. Maven encourages a standard project structure and provides a standard lifecycle for building projects.

Main features

• Build automation: compile, test, package and perform other build tasks.

• Dependency management: declare libraries and let Maven resolve them.

• Standard lifecycle: common phases provide a predictable build sequence.

• Plugins: specialized tasks are implemented through plugins.

• Repositories: artifacts and dependencies can be stored and retrieved from repositories.

• CI/CD support: Maven commands are easy to execute from automation servers.

Why Maven fits DevOps

DevOps values repeatability and automation. Maven lets a project describe its dependencies and build configuration in the repository, so a CI server can run a predictable command rather than relying on manual setup.

Memory trick: Maven = Project + Dependencies + Lifecycle + Plugins + Artifacts.

Maven's role in a pipeline

Git	CI	Maven	Artifact	Release
source	trigger	build + test	JAR/WAR	deploy

5. Maven Architecture & Core Concepts

The major pieces to understand

Concept	Meaning
POM	Project configuration file, normally named pom.xml.
Lifecycle	Ordered build process made up of phases.
Phase	A stage in a lifecycle, such as compile, test or package.
Goal	A specific task performed by a plugin.
Plugin	A component that provides goals used by Maven.
Artifact	A file produced or managed by Maven, such as a JAR.
Repository	Location used to store/retrieve Maven artifacts.

Phase vs Goal

A phase belongs to a lifecycle and represents a stage in the build. A goal is a specific plugin task. Maven maps lifecycle phases to plugin goals through its configuration and default bindings.

> Example: The `compile` phase is a lifecycle phase; a compiler plugin goal performs the actual compilation work associated with that phase.

Artifact coordinates

Maven commonly identifies an artifact using coordinates such as groupId, artifactId and version. These values help Maven locate and distinguish project artifacts.

Coordinate	Example	Purpose
groupId	com.example	Identifies the organization/project group.
artifactId	demo-app	Identifies the project/artifact name.
version	1.0.0	Identifies a particular project version.

6. pom.xml — Project Object Model

The pom.xml file is the central configuration file of a Maven project. It can describe the project identity, dependencies, plugins, properties, build settings and other project information.

Basic POM example

```
4.0.0 com.example demo-app 1.0.0
```

Element	Meaning
modelVersion	Version of the POM model being used.
groupId	Organization or project group identifier.
artifactId	Name of the project artifact.
version	Project version.
dependencies	Libraries required by the project.
build	Build-related configuration such as plugins and output settings.
properties	Reusable values such as Java version settings.

Dependency example

```
org.example example-library 1.2.3
```

Important: The example coordinates above are illustrative. In a real project, use the coordinates and versions required by that project's documentation.

7. Maven Standard Directory Structure

Maven encourages a conventional directory layout. Following conventions makes projects easier for developers and tools to understand.

```
demo-app/ ■■■ pom.xml ■■■ src/ ■■■ main/ ■ ■■■ java/ ■ ■■■ resources/ ■■■ test/ ■■■ java/
■■■ resources/
```

Directory	Typical purpose
src/main/java	Main Java source code.
src/main/resources	Main application resources/configuration.
src/test/java	Test source code.
src/test/resources	Resources used by tests.
target	Generated build output and artifacts; commonly recreated by builds.
pom.xml	Maven project configuration.

Why convention matters

- Developers know where to find source code and tests.
- Maven plugins can work with sensible defaults.
- CI systems can build projects without custom instructions for every folder.
- The project becomes easier to maintain and onboard new developers.

Exam point: Maven's convention-over-configuration approach means common projects can require less custom build configuration.

8. Maven Build Lifecycle

Maven organizes a build into lifecycles. For normal application builds, the default lifecycle is especially important. Maven also defines clean and site lifecycles.

Lifecycle	Purpose
default	Build, test, package, verify, install and deploy the project.
clean	Remove files generated by previous builds.
site	Generate project documentation/site information.

Default lifecycle — simplified view

validate	compile	test	package	verify	install	deploy
check	compile	test	artifact	checks	local repo	remote repo

Important rule

When you ask Maven to execute a later lifecycle phase, Maven normally executes the earlier phases in that lifecycle up to the requested phase. For example, reaching `package` requires the earlier lifecycle phases to run as appropriate.

> Example: `mvn package` does more than simply create a JAR/WAR; it runs the lifecycle up to the package phase, including the required earlier phases.

9. Maven Phases & Goals

Phase	Simple meaning
validate	Check that the project is valid and required information is available.
compile	Compile the main source code.
test	Compile and run tests as configured.
package	Create the distributable artifact.
verify	Perform additional checks on the results.
install	Install the artifact into the local repository.
deploy	Publish the artifact to a remote repository.

What is a goal?

A Maven goal is a specific task provided by a plugin. Goals can be invoked directly or can be bound to lifecycle phases. This is how Maven combines a high-level lifecycle with concrete build operations.

Concept	Example idea
Lifecycle phase	package
Plugin	A plugin responsible for compilation or testing.
Plugin goal	A concrete task such as compiling source code.

Phase and goal — easy analogy

Phase = what stage? → “We are at the testing stage.”
Goal = what task? → “Run this plugin task to perform the test operation.”

Exam question

Q: Why are Maven plugins important?
A: Plugins provide the concrete goals that perform many operations in the Maven build process.

10. Dependencies & Repositories

A dependency is a library or component that a project needs for compilation, testing or runtime. Maven allows dependencies to be declared in the POM instead of manually managing library files.

Why dependency management matters

- Keeps dependency versions explicit.
- Automatically resolves required artifacts.
- Can also resolve transitive dependencies required by another dependency.
- Reduces manual copying of library files.
- Makes automated builds easier to reproduce.

Repository types

Repository	Role
Local repository	A cache/storage area on the machine for downloaded or locally installed artifacts.
Remote repository	A repository hosted elsewhere that can provide or receive artifacts.
Central repository	A widely used public source for many common Maven artifacts.

Transitive dependencies

Suppose project A depends on library B, and B depends on library C. Maven can resolve C as a transitive dependency, subject to dependency configuration and version resolution rules.

> Memory: Your project may directly declare A, while Maven can discover the libraries required by A's dependencies.

Dependency declaration pattern

```
... ... ...
```

11. Maven Plugins & Profiles

Plugins

Maven plugins provide goals that perform specific operations. Build, compiler, testing, packaging and reporting tasks can be implemented or extended through plugins.

Plugin area	Typical responsibility
Compiler	Compile source code.
Testing	Run configured tests.
Packaging	Create or process artifacts.
Reporting	Generate reports or documentation.

Profiles

A Maven profile lets a project define alternative configuration for different environments or situations. Profiles can be activated in several ways, depending on configuration.

```
production
```

Where profiles can be useful

- Different development, testing and production configurations.
- Optional build settings for special environments.
- Environment-specific properties or plugin configuration.

Caution: Profiles should be designed clearly. Too many environment-specific differences can make builds harder to understand and reproduce.

12. Maven Commands & Practical Workflow

Command	Purpose
mvn validate	Validate project configuration.
mvn compile	Compile main source code.
mvn test	Run tests.
mvn package	Build the distributable artifact.
mvn clean	Remove previous generated build output.
mvn verify	Run verification checks after packaging.
mvn install	Install artifact into the local Maven repository.
mvn deploy	Publish artifact to a configured remote repository.

Typical developer workflow

1. Create or clone a Maven project containing pom.xml.
2. Add required dependencies to pom.xml.
3. Write source code and tests in the standard directories.
4. Run Maven to compile and test the project.
5. Package the application into an artifact.
6. Commit the project configuration and source code to version control.
7. Let CI run the same Maven build automatically.

Common examples

```
mvn clean package mvn test mvn verify mvn clean install
```

Remember: `clean` is a lifecycle goal/phase from the clean lifecycle, while `package` belongs to the default lifecycle.

13. Maven in CI/CD + Final Revision

Maven inside a CI/CD pipeline

Step	Example action
1. Developer	Writes code and tests.
2. Git	Pushes the code to a repository.
3. CI server	Detects the change and starts a pipeline.
4. Maven	Resolves dependencies and runs the configured build/test lifecycle.
5. Artifact	A JAR/WAR or other build output is produced.
6. Quality gates	Tests/checks determine whether the build is acceptable.
7. Deploy	A later pipeline stage releases the artifact to an environment.

Advantages of Maven

- Standard project structure and lifecycle.
- Automatic dependency management.
- Repeatable command-line builds.
- Plugin-based extensibility.
- Easy integration with CI/CD tools.
- Artifact and repository management.

Quick Viva Questions

What is Maven?

A build automation and project management tool, widely used for Java projects.

What is pom.xml?

The Project Object Model file containing Maven project configuration.

What is a dependency?

A library/component required by the project.

What is a lifecycle?

An ordered set of build phases.

What is a phase?

A stage of a Maven lifecycle, such as compile, test or package.

What is a goal?

A specific task provided by a Maven plugin.

What does mvn package do?

Runs the default lifecycle up to package and creates the configured artifact.

What does mvn install do?

Runs the build through install and places the artifact in the local repository.

Why Maven in DevOps?

It makes Java builds standardized, automated and suitable for CI/CD.

FINAL MEMORY MAP

POM → Dependencies → Lifecycle → Phases → Plugins/Goals → Artifact → Repository → CI/CD

Poster-ready line: “Maven turns source code into a repeatable, automated build.”